Improved Streaming Edge Coloring

Shiri Chechik[1] **Hongyi Chen**[2] Tianyi Zhang[3]

[1]Tel Aviv University

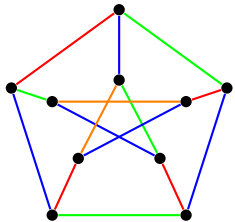
[2]Nanjing University

[3]ETH Zürich

ICALP 2025

Edge Coloring Problem

- **Input:** An undirected graph $G = (V, E)$.
- **Output:** A coloring $c : E \to \{1, 2, \ldots, k\}$ s.t. adjacent edges receive different colors.
- The minimum number of colors needed is the **edge chromatic number** $\chi'(G)$.



Valid 4-edge-coloring of a 3-regular graph ($\chi'(G) = 4$).

Edge Coloring Problem

- **Input:** An undirected graph $G = (V, E)$.
- **Output:** A coloring $c : E \to \{1, 2, \ldots, k\}$ s.t. adjacent edges receive different colors.
- The minimum number of colors needed is the **edge chromatic number** $\chi'(G)$.
- $\chi'(G) \geq \Delta$, where $\Delta = \max\{\deg(v)\}$
- $\chi'(G) \leq \Delta$ in simple bipartite graphs, $\chi'(G) \leq \Delta + 1$ in simple general graphs [Vizing, 1964]

But how do we compute such a coloring... in the streaming model?

The W-Streaming Model

- **Input**: A stream of edges (u, v) of graph $G = (V, E)$, where $n = |V|$ and $m = |E|$.
- **Single-pass only**.
- **Memory**: $\tilde{O}(n)$ (sublinear in m, cannot store whole graph).
- However, even writing down a full edge coloring takes $\Omega(m)$ space.
- **W-Streaming vs. Classical Streaming**: Allows a *write-only* output stream.
  - Classical: output only at end.
  - W-Streaming: output incrementally on the fly.
- **Goal**: Produce a valid edge coloring using as few colors as possible.

Background: Prior Work

Algorithm Type	# Colors	Space	Reference
Randomized	Δ^2	$\widetilde{O}(n)$	[Behnezhad et al., 2019]
Randomized	$O(\Delta^2/s)$	$\widetilde{O}(ns)$	[Charikar and Liu, 2021]
Deterministic	$O(\Delta^2/s)$	$O(ns)$	[Ansari et al., 2022]
Deterministic	$\widetilde{O}(\Delta^2/s^2)$	$\widetilde{O}(ns)$	[Ghosh and Stoeckl, 2024]
Randomized	$\widetilde{O}(\Delta^{1.5})$	$\widetilde{O}(n)$	[Chechik et al., 2024]
Randomized	$\widetilde{O}(\Delta^{1.5}/s)$	$\widetilde{O}(ns)$	[Behnezhad and Saneian, 2024]

Question:

Can we achieve $O(\Delta)$ coloring using $\tilde{O}(n)$ space — randomly or even deterministically?

Our Results

Main Theorem (Randomized)

For any $\varepsilon > 0$, there exists a randomized W-streaming algorithm using:

- $\tilde{O}_\varepsilon(n)$ space
- $\tilde{O}(\Delta^{4/3+\varepsilon})$ colors (in expectation)

Main Theorem (Deterministic)

For any $\varepsilon > 0$, there exists a deterministic W-streaming algorithm using:

- $\tilde{O}_\varepsilon(n)$ space
- $\tilde{O}(\Delta^{4/3+\varepsilon})$ colors
- Slightly higher log factors in space and color count

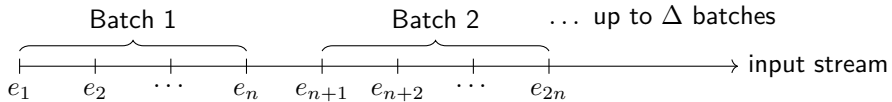
Warm-up: Deterministic Δ^2 Coloring

- For simplicity, consider bipartite graphs $G = (L \cup R, E)$. The general case can be reduced with a constant-factor overhead [Ghosh and Stoeckl, 2024].
- Each vertex maintains its current degree $\deg(v)$.
- When an edge (u, v) arrives, assign color: $(\deg(u),\ \deg(v))$
- Then assign $\deg(u) \leftarrow \deg(u) + 1, \deg(v) \leftarrow \deg(v) + 1$

Warm-up: $\widetilde{O}(\Delta^{1.5})$ Randomized Algorithm

[Behnezhad and Saneian, 2024, Chechik et al., 2024]

- Process edges in *batches of* n *edges*.
- Case 1: If the subgraph formed by the batch has maximum degree $\leq \Delta^{1/2}$:
  - Use $\Delta^{1/2}$ fresh colors per batch
  - Total number of colors used over all Δ batches: $\Delta \cdot \Delta^{1/2} = \Delta^{3/2}$



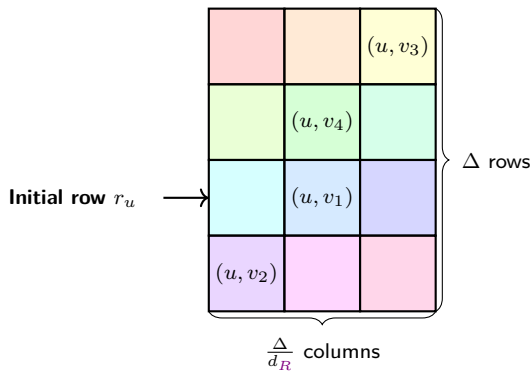
Warm-up: $\widetilde{O}(\Delta^{1.5})$ Randomized Algorithm

[Behnezhad and Saneian, 2024, Chechik et al., 2024]

- Case 2: Assume in each batch, every vertex in R has degree $d_R > \Delta^{1/2}$:
  - Maintain a color table $\mathcal{C}[i, j]$ of size $\Delta \times (\Delta/d_R)$.
  - u determines the row index, and v determines the column index.

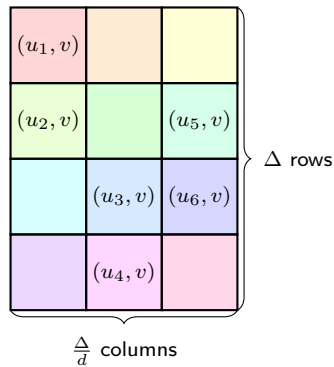
Color Table for Fixed Vertex u

- Each vertex $u \in L$ draws an initial row offset $r_u \sim [\Delta]$.
- When an edge (u, v) arrives, move to the next row.
- Exactly one color from each row is used for u.



Color Table for Fixed Vertex v

- For edges (u_i, v) within the same batch:
  - All colors lie in the same column.
  - Since each u_i draws a random offset r_{u_i}, $(1 - 1/\Delta)^{d_R} \geq 1/e$ of these u_i get distinct colors in expectation.
- Coloring a constant fraction of edges is sufficient: uncolored edges can be sent to a virtual substream and handled recursively.
- When v appears in a new batch, move to the next column.
- v appears in at most $\frac{\Delta}{d}$ batches (columns).

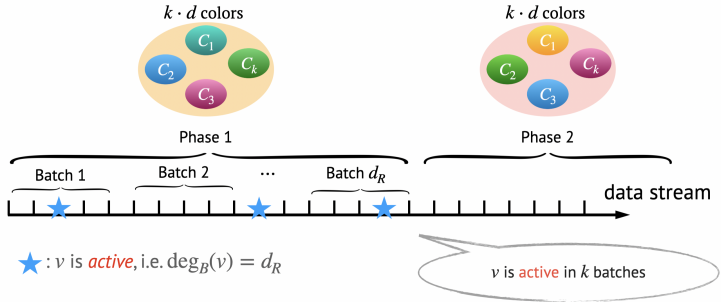


Technical Overview: Breaking the $\Delta^{1.5}$ Barrier

- If every vertex in L has degree d_L and every vertex in R has degree d_R, the color table method uses
$$\frac{\Delta}{d_L} \cdot \frac{\Delta}{d_R} = \frac{\Delta^2}{d_L d_R}$$
colors. This breaks the $\Delta^{1.5}$ barrier when degrees are balanced.
- Bottleneck: Highly unbalanced batches — e.g., $d_L = 1$, $d := d_R = \Delta^{1/2}$ — lead to $\Omega(\Delta^{1.5})$ colors.
- Our approach: a multi-layer scheme + random package allocation.

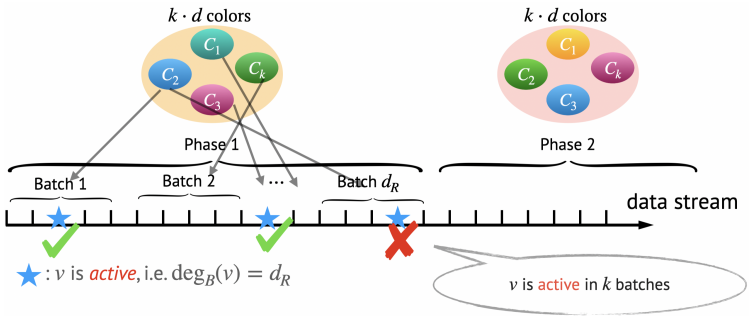
$O(\Delta^{5/4})$ Coloring for Unbalanced Batches

- Partition the Δ batches into Δ/d **phases**, each consisting of d consecutive batches.
- Case 1: If each **phase** has maximum degree $k \cdot d$ ($k \leq \Delta^{1/4}$)
  - In principle, each phase can be colored using $O(kd)$ different colors.
  - Total number of colors used over all Δ/d phases : $\Delta/d \times kd = k\Delta$



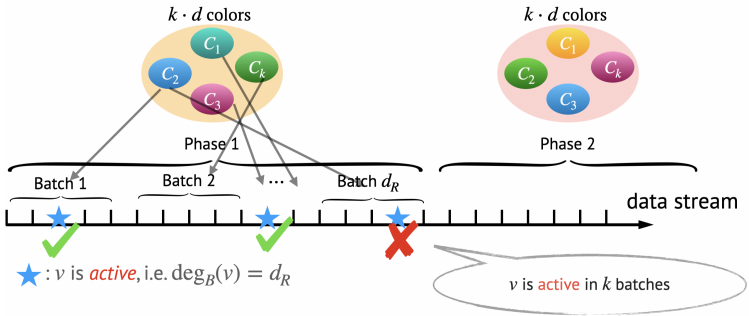
$O(\Delta^{5/4})$ Coloring for Unbalanced Batches

- **Goal**: color each phase using $O(k \cdot d)$ colors.
- Partition the color package into k palettes $C_1, \ldots, C_k$, each of size d.
- Each batch randomly selects one palette C_i to use.
- Observation 1:
  - each vertex $v \in R$ appears in k batches
  - each batch chooses a palette uniformly at random
  - $\geq (1 - 1/k)^k \approx 1/e$ of the "active" batches use distinct palettes.



$O(\Delta^{5/4})$ Coloring for Unbalanced Batches

- **Goal**: color each phase using $O(k \cdot d)$ colors.
- Observation 2: Each vertex $v \in R$ can maintains the set of palettes used during the current phase.
  - At most $nd/(dk) = n/k$ vertices need to be tracked.
  - Each such vertex uses $O(k)$ space.

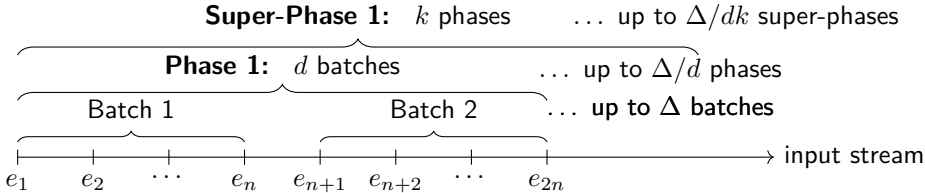


$O(\Delta^{5/4})$ Coloring for Unbalanced Batches

- **Goal**: color each phase using $O(k \cdot d)$ colors.
- For each edge (u, v):
  - Vertex u selects a color from the chosen palette using:
    - a degree counter, and
    - a vertex-specific random shift.
  - **Challenge:**
    - Using a separate counter for each vertex and each palette require $O(k \cdot n)$ space.
  - **Key observation:**
    - In each batch, all degrees in L increase synchronously by 1.
    - So counters can be shared across vertices.
  - **Solution:** Maintain one global counter per palette.

$O(\Delta^{5/4})$ Coloring for Unbalanced Batches

- Case 2: If some **phase** has maximum degree $k \cdot d$ ($k > \Delta^{1/4}$), using a fresh palette each phase is too expensive.
- Group k phases into a **super-phase**



- Allocate a super-package P_i of size Δ to each super-phase:
  - Each super-package is divided into Δ/dk packages of size dk.
  - Each phase randomly selects a package from its super-package, as before.
- Vertices in R now track used packages (one per phase)
- Total colors: Δ per super-phase $\times$ Δ/kd super-phases $= O(\Delta^{5/4})$ (Recall $d = \Delta^{1/2}$)

Technical Overview: Achieving $\tilde{O}_\varepsilon(\Delta^{4/3})$

- Set $k = \Delta^\varepsilon$ and extend to a $1/\varepsilon$-layer scheme.
  - This handles the extreme case $(d_L = 1)$ using $O_\varepsilon(\Delta^{1+\varepsilon})$ colors.
- Generalizing to arbitrary degrees (d_L) gives color count: $d_L \cdot O_\varepsilon(\Delta^{1+\varepsilon})$.
- Meanwhile, the color table method yields $O(\Delta^2/(d_L \cdot d_R))$ colors.
- For general graphs, solve each pair $d_L, d_R \in \{2^i : i = 0, 1, \ldots, \lceil \log \Delta \rceil\}$ separately, then combine the results.
- Balancing the two bounds minimizes total colors at $\tilde{O}_\varepsilon(\Delta^{4/3})$.

Thank you!

References I


Ansari, M., Saneian, M., and Zarrabi-Zadeh, H. (2022).
Simple streaming algorithms for edge coloring.
In *30th Annual European Symposium on Algorithms (ESA 2022)*.
Schloss-Dagstuhl-Leibniz Zentrum für Informatik.


Behnezhad, S., Derakhshan, M., Hajiaghayi, M., Knittel, M., and Saleh, H. (2019).
Streaming and massively parallel algorithms for edge coloring.
In *27th Annual European Symposium on Algorithms (ESA 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.


Behnezhad, S. and Saneian, M. (2024).
Streaming edge coloring with asymptotically optimal colors.
In *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

References II


Charikar, M. and Liu, P. (2021).
Improved algorithms for edge colouring in the w-streaming model.
In *Symposium on Simplicity in Algorithms (SOSA)*, pages 181–183. SIAM.


Chechik, S., Mukhtar, D., and Zhang, T. (2024).
Streaming Edge Coloring with Subquadratic Palette Size.
In Bringmann, K., Grohe, M., Puppis, G., and Svensson, O., editors, *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, volume 297 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 40:1–40:12, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

References III



Ghosh, P. and Stoeckl, M. (2024).
Low-Memory Algorithms for Online Edge Coloring.
In Bringmann, K., Grohe, M., Puppis, G., and Svensson, O., editors, *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, volume 297 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 71:1–71:19, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.